Module 1 — Viva Speech (Dinuka)

Authentication, Registration and Service Provider Invoicing. Roughly 5 minutes at a normal speaking pace. Square brackets are delivery notes, not words to say.

Opening — what I own (about 30 seconds)

Good morning. My name is Dinuka, and I was responsible for **Module 1 of HomeHero — Authentication, Registration, and Service Provider Invoice Generation**.

My module is the front door of the system. Every user who enters HomeHero passes through my code first — whether they are creating an account, logging in, or recovering a forgotten password. Once a user is authenticated and sent to the right dashboard, my responsibility hands over to whichever member owns that section.

I was also assigned a second feature separately: **invoice generation for service providers**.

The features I developed (about 1 minute)

[Count them off — this is the list they're marking.]

I built **six** main features.

One — client registration, including selecting their location on a map.

Two — service provider registration, which is a two-stage process with document uploads.

Three — login, with role-based redirection to five different destinations.

Four — logout and session management.

Five — forgot password and password reset, using secure expiring links.

Six — invoice generation, where a provider creates a PDF invoice for a completed job.

System flow — registration and login (about 1 minute 15)

Let me walk through the main flow.

A visitor chooses whether they are registering as a **client** or a **service provider**.

A client enters their details and places a marker on a map for their address. Before I create anything, I check that the username is not taken and the email is not already registered. Then the password is hashed, a unique token is generated, and the user record, the client profile and the location are all written **inside one transaction**. If any part fails, nothing is saved — we never end up with a half-created account. The client's account is active immediately and they receive a welcome email.

A service provider is different. They complete two stages: personal and service details first, then their verification documents — a face photograph, both sides of their NIC, and a police report. They must also accept the terms and conditions.

[Slow down here.]

The important difference is that a provider account is **not active on creation**. It is saved with verification status **Pending**, and the applicant is sent to a waiting page. They cannot access provider features until the Verification Admin — Vihas's module — approves them.

When a user logs in, I check the password against the stored hash, then check whether the account is deactivated, and then whether there is an active ban. A banned user is refused with the reason and, for a temporary ban, the date it ends. Only after all of those checks do I create the session and redirect the user by role — a client to the client home page, a

verified provider to their dashboard, a pending provider back to the waiting page, and the two admins to their own dashboards.

Security decisions (about 1 minute)

There are four security decisions I would like to highlight.

First, passwords. They are hashed with **bcrypt**, using twelve salt rounds. The plain password is never stored and never logged. When someone logs in, I compare hashes — I never decrypt anything, because a bcrypt hash cannot be reversed.

Second, the unique user token. Every client and provider gets a six-character token that mixes uppercase, lowercase and digits. I generate it with a **cryptographic** random function, and I guarantee it contains at least one of each character type. Before assigning it I check the database for a collision and regenerate if needed. This token is what other members use to identify users — complaints, admin searches, booking tables — so the internal database ID is never exposed.

Third, sessions. When a user logs in I create a session row in the database, and the JWT the browser holds is tied to that row. This matters: it means a session can be **individually revoked**. When Vihas's module bans a user, their active sessions are invalidated immediately rather than staying valid until the token expires on its own.

Fourth, the password reset link. The token in the email is random and expiring, and — this is the key part — **I do not store that token.** I store only a SHA-256 hash of it. So even if someone read the database, they could not reconstruct a working reset link. The token is single-use and is marked used the moment the password changes.

One more detail: on the forgot-password page, if the email does not exist in the system, I return **the same success message** rather than an error. Otherwise the page becomes a way for an attacker to discover which email addresses have accounts.

Validation (about 45 seconds)

I validated at **three levels**.

On the frontend, for immediate feedback — required fields, matching passwords, valid formats.

On the backend, every registration and login request passes through a schema before it reaches my logic. Usernames must be three to thirty characters of letters, numbers or underscore. Phone numbers must match the **Sri Lankan** format. Passwords must be at least eight characters and contain both a letter and a number. And the password and confirmation must match — checked again on the server, not just in the browser.

And for file uploads there is a third layer. The NIC images and face photograph must be JPG or PNG. The police report may also be a PDF. Every file has a maximum size, and the type is checked from the file itself rather than trusting its name. The uploaded documents are stored in a **private** folder, not in the public web directory — only the Verification Admin can view them.

Invoice generation (about 45 seconds)

My second feature is invoicing.

Once a job is completed, the provider can generate **one** invoice for it. Before anything happens I check three things: the booking exists, it belongs to *this* provider, and its status is Completed.

The invoice details are filled in automatically from the booking. The only field the provider touches is the amount — and how that behaves depends on how the client paid.

If the client paid by card, the amount is read directly from Visal's payment record and is **locked**. The provider cannot type or change it. This guarantees the invoice always matches HomeHero's own record of that payment.

If the client paid cash, HomeHero was never involved in the money, so the field starts empty and the provider enters the agreed amount themselves.

The system then generates a formatted PDF, saves it to private storage, and links it permanently to that booking. After that, the menu changes from *Generate Invoice* to *Download Invoice* — and if they try to generate a second one, the request is rejected. Downloads always return the same stored file, so an invoice can never be quietly rewritten after the fact.

Emails and integration (about 30 seconds)

My module sends **two** emails directly: a welcome email when a client registers, and the password reset link.

I also built the **shared email service** that the other members' modules use for their own messages — verification approval and rejection, membership confirmations, ban notices and complaint acknowledgements. Each member writes their own wording; the sending, logging and retry handling is mine.

For integration, my module hands over to every other member: authenticated users go to Tharinsa's client pages, Maheli's provider dashboard or Vihas's admin panels. My invoice feature reads the locked amount from Visal's payment records, and the generated file is displayed by Maheli's Completed Jobs page and Vihas's SP Tracking page.

Closing (about 15 seconds)

So in summary: my module controls who gets into HomeHero and what they can reach, protects passwords and reset links with hashing rather than storage, ties every session to a revocable database record, and produces tamper-proof invoices for completed jobs.

Thank you — I'm happy to take any questions.

60-second version

[If they cut you short.]

I was responsible for authentication, registration and service provider invoicing.

Clients register with a map-selected address and become active immediately. Providers register in two stages with document uploads, and stay Pending until the Verification Admin approves them — they cannot use provider features before that. Login checks the password hash, then account status, then bans, and redirects by role.

Passwords are hashed with bcrypt at twelve rounds. Every user gets a cryptographically random six-character token so their database ID is never exposed. Sessions are stored in the database so they can be revoked instantly when someone is banned. Password reset links are stored only as a hash, expire, and are single-use.

For invoicing, a provider generates one PDF per completed job. For card payments the amount is locked to the recorded payment and cannot be edited; for cash the provider enters it. Once generated it cannot be regenerated, only downloaded again.